Arquitectura de despliegue

Portal institucional

Agencia ITRC

Documento técnico de referencia

Dirigido a equipo de infraestructura y administradores del sitio

Contenido del documento:

- Migración de WordPress a JAMstack (Astro + Sveltia CMS)
- Arquitectura de despliegue sobre servidor propio (Ubuntu + nginx)
- Flujo de trabajo Git con múltiples webmasters
- Autenticación y administración de usuarios
- Manejo de contenido estático y binarios

Versión 1.0 | 2026

Índice

1. Resumen ejecutivo **2**
1.1. Componentes principales . . . 2

2. Diagrama de arquitectura **3**

3. Flujo de publicación paso a paso **4**
3.1. Escenario 1: Webmaster edita con VS Code . . . 4
3.2. Escenario 2: Webmaster edita con Sveltia CMS . . . 4

4. Configuración del servidor (Ubuntu + nginx) **5**
4.1. Requisitos mínimos del servidor . . . 5
4.2. Usuario de despliegue . . . 5
4.3. Configuración nginx mínima . . . 5

5. Autenticación y administración de usuarios **7**
5.1. Modelo de cuentas . . . 7
5.2. Flujo de autenticación (OAuth con GitHub) . . . 7
5.3. Alta de un webmaster nuevo . . . 7
5.4. Roles sugeridos . . . 7
5.5. Baja de un webmaster . . . 8
5.6. Buenas prácticas de seguridad . . . 8

6. Consideraciones operativas **9**
6.1. Manejo de binarios . . . 9
6.2. Rollback (revertir un cambio) . . . 9
6.3. Monitoreo y logs . . . 9
6.4. Respaldos . . . 9

Referencias **10**

1 Resumen ejecutivo

El portal institucional de la Agencia ITRC migró desde una arquitectura WordPress (dinámica, basada en base de datos MySQL y PHP) hacia una arquitectura **JAMstack** basada en el generador de sitios estáticos Astro y el gestor de contenido Sveltia CMS. El portal se despliega sobre el servidor institucional del ITRC (Ubuntu con nginx).

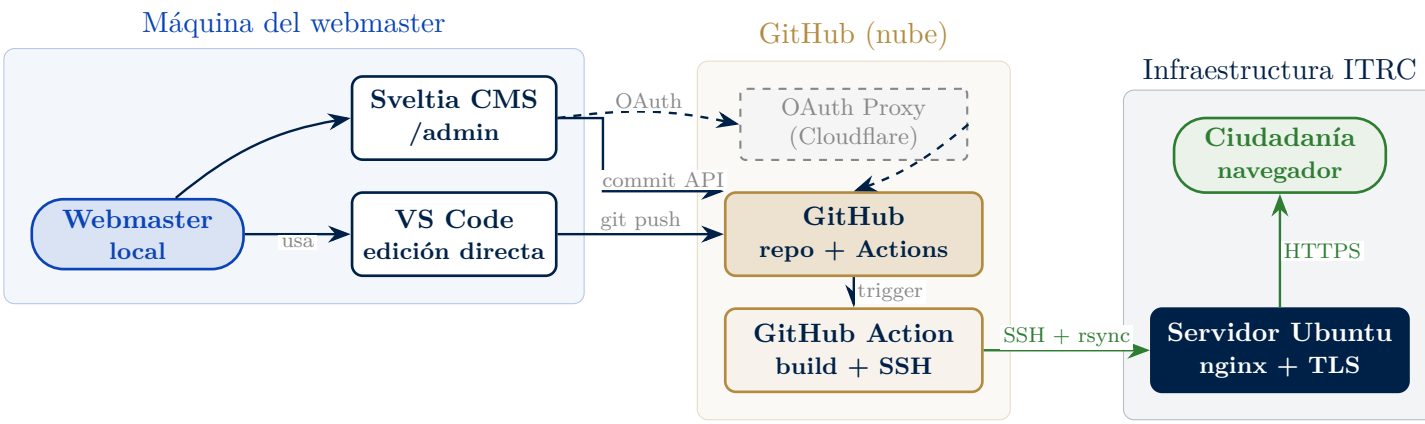
Cambios clave respecto al modelo anterior

- **Sin base de datos**: todo el contenido es archivos JSON y Markdown versionados en Git.
- **Sin PHP**: el servidor únicamente sirve archivos estáticos (HTML, CSS, JS, PDF, imágenes).
- **Git como fuente de verdad**: cada cambio es un *commit* con trazabilidad y posibilidad de *rollback*.
- **Despliegue automatizado**: cada `git push` a la rama principal dispara la publicación en el servidor.
- **Múltiples webmasters**: cada uno con cuenta GitHub propia y acceso auditado.

1.1 Componentes principales

Componente	Rol
Astro	Generador de sitio estático. Transforma archivos JSON/MD en HTML.
Sveltia CMS	Panel de administración visual (editor de contenido) que commitea al repo Git.
GitHub	Almacenamiento del código, control de versiones, y orquestador de despliegues.
GitHub Actions	Pipeline CI/CD que construye y sube el sitio al servidor.
Ubuntu + nginx	Servidor institucional que sirve los archivos al usuario final.
OAuth Proxy	Intermediario que permite al CMS autenticarse contra GitHub.

2 Diagrama de arquitectura



Cómo leer el diagrama

Las flechas **azules continuas** representan acciones del usuario o del sistema. Las **azules punteadas** representan handshakes de autenticación. Las flechas **verdes** representan el camino final del contenido publicado hasta la ciudadanía.

3 Flujo de publicación paso a paso

3.1 Escenario 1: Webmaster edita con VS Code

Aplica cuando el webmaster realiza cambios estructurales o bulk edits que no son prácticos desde el CMS.

1. El webmaster ejecuta `git pull` en su copia local para sincronizarse con los cambios más recientes.
2. Abre el proyecto en VS Code y modifica archivos: JSON de contenido, imágenes en `public/`, o código del portal.
3. Ejecuta `npm run dev` localmente para previsualizar cambios en `http://localhost:4321`.
4. Satisfecho con el resultado, ejecuta `git commit` y `git push`.
5. GitHub recibe el push y dispara automáticamente el workflow de GitHub Actions.
6. El Action construye el sitio (`npm run build`) y sube los archivos al servidor vía SSH+rsync.
7. nginx comienza a servir la nueva versión. Tiempo total: 2 a 4 minutos.

3.2 Escenario 2: Webmaster edita con Sveltia CMS

Aplica para el flujo cotidiano: publicar noticias, actualizar documentos, cambiar banners, modificar textos.

1. El webmaster navega a `https://portal.itrc.gov.co/admin` en su navegador.
2. Hace clic en *Login with GitHub*. El flujo OAuth lo lleva a autenticarse con su cuenta.
3. Una vez autenticado, ve el panel del CMS con las colecciones (noticias, páginas, slider, etc.).
4. Realiza los cambios necesarios usando el editor visual.
5. Hace clic en *Publish*. Sveltia CMS crea automáticamente un commit en el repo con la autoría de su cuenta GitHub.
6. A partir del paso 5 del escenario 1: GitHub Action dispara, construye, despliega.

Trazabilidad

Cada commit (ya sea hecho por VS Code o por Sveltia CMS) queda registrado en el historial Git con autor, fecha, y descripción. Esto permite auditar quién publicó qué y cuándo, sin necesidad de sistemas externos.

4 Configuración del servidor (Ubuntu + nginx)

4.1 Requisitos mínimos del servidor

Componente	Versión sugerida
Sistema operativo	Ubuntu Server 22.04 LTS o superior
nginx	1.18 o superior
Espacio en disco	Mínimo 10 GB (actual: ∼4 GB, con margen)
RAM	Mínimo 1 GB (el sitio no requiere más que esto)
CPU	Cualquier CPU moderno (el servidor solo sirve estáticos)
Puertos	80 y 443 abiertos al público; 22 solo para administración

4.2 Usuario de despliegue

Se crea un usuario específico para los despliegues automatizados, sin privilegios `sudo`:

```
sudo useradd -m -s /bin/bash portal-deploy
sudo mkdir -p /var/www/portal.itrc.gov.co
sudo chown portal-deploy:www-data /var/www/portal.itrc.gov.co
sudo chmod 755 /var/www/portal.itrc.gov.co
```

La clave pública SSH del workflow de GitHub Actions se añade a `/home/portal-deploy/.ssh/authorized_keys`.

4.3 Configuración nginx mínima

Archivo `/etc/nginx/sites-available/portal.itrc.gov.co`:

```
server {
    listen 80;
    server_name portal.itrc.gov.co;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name portal.itrc.gov.co;

    root /var/www/portal.itrc.gov.co;
    index index.html;

    ssl_certificate     /etc/letsencrypt/live/portal.itrc.gov.co/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/portal.itrc.gov.co/privkey.pem;

    # Headers de seguridad
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;
    add_header Strict-Transport-Security "max-age=31536000" always;

    # Compresión
    gzip on;
    gzip_types text/plain text/css application/javascript application/json;

    # Cache para assets estáticos
    location /_astro/ {
```

```
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
    location /documentos/ {
        expires 7d;
        add_header Cache-Control "public";
    }

    # Rutas del portal
    location / {
        try_files $uri $uri.html $uri/index.html =404;
    }
}
```

El certificado TLS se obtiene gratuitamente con Let's Encrypt vía `certbot`.

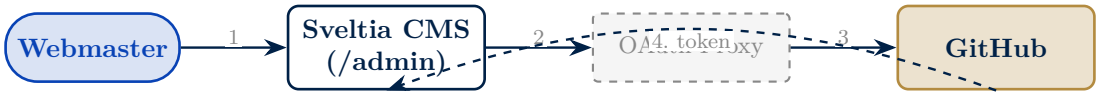
5 Autenticación y administración de usuarios

5.1 Modelo de cuentas

Cada webmaster de la institución cuenta con una **cuenta personal de GitHub**. No se comparten credenciales entre personas. Las razones son:

- **Trazabilidad**: todos los commits en el historial quedan asociados a un nombre real.
- **Seguridad**: si una cuenta se compromete, se revoca su acceso individualmente sin afectar al resto.
- **Escalabilidad**: añadir o retirar personas es cuestión de un clic, sin pasar por el equipo técnico cada vez.
- **Auditoría**: cualquier auditoría posterior puede reconstruir quién hizo qué con precisión.

5.2 Flujo de autenticación (OAuth con GitHub)



1. El webmaster entra al CMS y solicita login.
2. El CMS redirige al OAuth Proxy (aloja credenciales de la app OAuth registrada en GitHub).
3. El Proxy redirige a GitHub para que el usuario autorice.
4. GitHub entrega un token al CMS via el Proxy. El CMS queda autenticado durante la sesión (típicamente 8 horas).

5.3 Alta de un webmaster nuevo

El procedimiento, a cargo del coordinador técnico:

1. El webmaster crea (o ya tiene) una cuenta personal en github.com. Se recomienda usar el correo institucional.
2. El coordinador accede al repositorio del portal en GitHub, en la sección *Settings* → *Collaborators*, y añade al nuevo webmaster con el rol **Write**.
3. GitHub envía una invitación por correo. El webmaster la acepta.
4. El webmaster prueba entrar a `https://portal.itrc.gov.co/admin` y hacer login. Si llega al dashboard, está listo.

5.4 Roles sugeridos

Rol GitHub	Puede hacer	Para quién
Admin	Todo + gestionar colaboradores	1–2 coordinadores técnicos
Maintain	Push + gestionar issues	Coordinadores de contenido
Write	Push a main (suficiente para CMS)	Webmasters regulares
Read	Solo lectura	Auditoría, visores externos

5.5 Baja de un webmaster

Al retirar a una persona:

- Se remueve su acceso en *Settings* → *Collaborators* → *Remove*.
- Los commits previos quedan en el historial (no se eliminan, por auditoría).
- No se revoca la aplicación OAuth: está asociada al portal, no a la persona.

5.6 Buenas prácticas de seguridad

- Exigir **autenticación en dos pasos (2FA)** en todas las cuentas que tengan acceso al repo.
- Revisar trimestralmente la lista de colaboradores y retirar accesos inactivos.
- El *Client Secret* del OAuth App nunca debe aparecer en código ni en commits.
- Ante sospecha de cuenta comprometida: remover acceso inmediato, cambio de contraseña y rotación de 2FA por parte del usuario, y reincorporación controlada.

6 Consideraciones operativas

6.1 Manejo de binarios

Los archivos binarios del portal (PDFs, hojas de cálculo, imágenes, audios, videos) residen en el directorio `public/documentos/` del repositorio y están **versionados junto al código**. Esto implica:

- Cualquier archivo subido por Sveltia CMS se commitea automáticamente al repo.
- El clon inicial del repositorio es del orden de ∼3.5 GB. Esto es una sola vez; los `pull` posteriores solo transmiten cambios.
- La ventaja: una única fuente de verdad para contenido y binarios.
- La ruta de servido se controla por la variable `DOCS_BASE_URL` en el código. Para despliegue en servidor propio, el valor es `/documentos`.

6.2 Rollback (revertir un cambio)

Si se publica un cambio erróneo:

1. Identificar el commit problemático con `git log -oneline`.
2. Ejecutar `git revert <hash>` — esto crea un nuevo commit que deshace los cambios.
3. `git push`. El GitHub Action despliega la versión restaurada en 2–4 minutos.

Nunca forzar el historial

No usar `git push -force` sobre la rama principal. El historial es auditable y debe preservarse. La forma correcta de deshacer cambios publicados es siempre con `git revert`, que deja evidencia de la corrección en el log.

6.3 Monitoreo y logs

- **Logs de nginx**: `/var/log/nginx/access.log` y `error.log` en el servidor. Rotación semanal estándar.
- **Estado de despliegues**: pestaña *Actions* del repositorio en GitHub, cada workflow se registra con éxito/fallo.
- **Disponibilidad**: monitoreo externo opcional (uptime pings) hacia `portal.itrc.gov.co`.

6.4 Respaldos

El modelo provee respaldo implícito:

- El repositorio en GitHub es un backup por sí mismo (replicado en infraestructura GitHub).
- Cada webmaster con su clon local tiene otra copia completa.
- El historial Git permite restaurar cualquier versión anterior por fecha o commit.
- Para infraestructura crítica, el coordinador puede configurar adicionalmente un `git clone -mirror` periódico en un servidor institucional secundario.

Referencias

- Astro: https://astro.build
- Sveltia CMS: https://github.com/sveltia/sveltia-cms
- nginx documentation: https://nginx.org/en/docs/
- Let's Encrypt / Certbot: https://certbot.eff.org
- GitHub Actions: https://docs.github.com/actions
- Manual del operador (portal ITRC): `docs/manual-operador/README.md` en el repo